

2026 LUOGU 非专业级别收容能力认证第一轮

(SCP-S1) 提高级 C++ 语言试题

认证时间：2026 年 8 月 16 日 14:30~16:30

考生注意事项：

- 试题纸共有 18 页，满分 100 分。请在洛谷作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。
- 本套试题难度高于一般的 CSP-S 初赛，其主要用途在于辅助考生进行自我评估，以检验当前阶段的学习成效，明晰自身优势与不足，而非单纯追求得分率。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 系统下本地测试函数式交互题，源程序名为 `scp.cpp`，源程序内引用了头文件 `scp.h`，题目下发交互库为 `grader.cpp`，则以下哪条编译命令可以输出正确的可执行文件 `scp`？（ ）
A. `g++ grader.cpp scp.h -o scp -O2 -std=c++14 -static`
B. `g++ grader.cpp scp.cpp -o scp -O2 -std=c++14 -static`
C. `g++ scp.h scp.cpp -o scp -O2 -std=c++14 -static`
D. `g++ scp.cpp scp.h -o scp -O2 -std=c++14 -static`
2. 关于最短路算法，以下说法中完全正确的是。（ ）
A. 只要图中没有负环，Dijkstra 算法就一定能求出正确的最短路径。
B. SPFA 算法在任何图上的最坏时间复杂度都比 Dijkstra 优秀。
C. 标准的 Dijkstra 算法（基于贪心，结点出队后不再更新）在包含负权边的图中可能会得到错误的结果。
D. Floyd 算法只能求任意两点间的最短路，无法判断图中是否存在负环。
3. 某算法的时间复杂度的递归式为 $T(n) = 2T(\sqrt{n}) + \Theta(\log n)$ ，则其渐进时间复杂度为？（ ）
A. $\Theta(\log n)$
B. $\Theta(\log \log n)$
C. $\Theta(\log^2 n)$
D. $\Theta(\log n \log \log n)$
4. 使用数 1,2,3,4,5 各一个以及 2 个加号、2 个减号可组成的后缀表达式的值最大为？（ ）
A. 9
B. 11
C. 12
D. 13

5. 运行如下 C++ 代码片段后, 关于数组 v 的状态, 下述说法**正确**的是? ()

```
std::vector<int> v = {9, 2, 7, 4, 5, 8, 1, 3, 6};  
std::nth_element(v.begin(), v.begin() + 3, v.end());
```

- A. $v[3]$ 的值必定是 4，且 $v[0]$ 到 $v[2]$ 的值必定依次是 1,2,3。
- B. $v[3]$ 的值必定是 4，且 $v[0]$ 到 $v[2]$ 包含 1,2,3 这三个数。
- C. $v[3]$ 的值必定是 4，且整个数组已经被完全排序。
- D. $v[0]$ 到 $v[3]$ 包含 1,2,3,4，且右侧的 $v[4]$ 到 $v[8]$ 必定已按从小到大排序。
6. 使用 `std::set` 维护有序集合时，设集合 s 中现有 n 个元素，则使用 `std::lower_bound(s.begin(), s.end(), x)` 查找第一个 $\geq x$ 的元素的时间复杂度为？（ ）
- A. $\Theta(\log n)$
- B. $\Theta(\log^2 n)$
- C. $\Theta(n)$
- D. $\Theta(n \log n)$
7. 以下选项中，哪个序列不可能是对长度为 6 的字符串 S 运行 KMP 算法得到的 next 数组（定义 $next[i]$ 为 $S[1 \dots i]$ 的最长相等真前后缀长度）？（ ）
- A. 0 0 0 1 2 3
- B. 0 0 0 1 2 1
- C. 0 1 0 1 2 1
- D. 0 1 2 0 1 2
8. 对 10 个互不相同且均在 $[0, 31]$ 之间的整数建立深度为 5（定义为叶子到根的距离）的 01-Trie，所可能得到的最多结点数和最少结点数之差为？（ ）
- A. 0
- B. 13
- C. 22
- D. 55
9. 使用数据结构维护序列区间信息 $a_l \oplus a_{l+1} \oplus \cdots \oplus a_r$ ，其中 $\oplus$ 为一般二元运算，下列说法正确的是？（ ）
- A. ST 表能 $\Theta(1)$ 回答区间询问的原因是利用了区间的重叠，要求 $\oplus$ 有交换律。
- B. 若使用线段树进行单点修改和区间查询， $\oplus$ 可以不满足结合律，但必须满足交换律。
- C. 基于前缀差分方式实现的树状数组，要询问区间信息，则要求 $\oplus$ 必须可逆（存在逆元）。
- D. 若使用平衡树维护序列区间信息， $\oplus$ 必须同时满足交换律和结合律，否则无法合并。
10. 对于一个包含 10 个顶点的无向简单图 G ，设其补图为 $\overline{G}$ ，下列说法正确的是？（ ）
- A. 若 G 不连通，那么 $\overline{G}$ 必然也不连通。
- B. 存在某个 G ，使得 G 和 $\overline{G}$ 都是一棵树。
- C. 存在某个 G ，使得 G 和 $\overline{G}$ 同时为二分图。
- D. 不存在 G ，使得 G 和 $\overline{G}$ 同时拥有欧拉回路。

11. 计算 $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{2026}$ 每项对 3 取模的值为? ()

A. $\begin{bmatrix} 1 & 2 \\ 2 & 0 \end{bmatrix}$

B. $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$

C. $\begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}$

D. $\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$

12. 有一个正十二面体, 共 20 个顶点、30 条棱, 每个面都是正五边形, 其棱长为 1, 顶点之间只能通过棱互相到达, 则所有 $\binom{20}{2}$ 个点对间最短路径长度之和为? ()

A. 420

B. 500

C. 600

D. 760

13. 有一排 10 盏灯, 初始时全部为关闭状态; 每次操作可以选定一个连续区间, 并将该区间内的灯亮灭状态反转, 则恰好 3 次操作后, 这 10 盏灯可能呈现出多少种不同的亮灭状态? ()

A. 120

B. 512

C. 848

D. 1024

14. 对于长度为 2026 的排列 p , 初始时对于所有位置 i 均有 $p_i = i$; 现在进行恰好 10^4 次操作, 每次操作可以交换相邻两个位置上的数, 则操作完成后得到的新排列 p' 的最长上升子序列长度最短是多少? ()

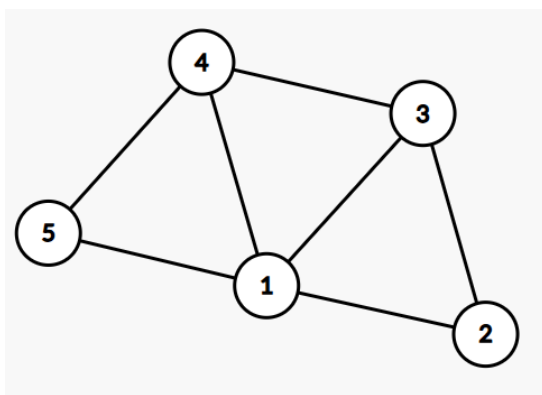
A. 110

B. 183

C. 186

D. 187

15. 对于如下图所示的无向图 G , 在所有将边赋予 $[1, 7]$ 中互不相同的整数边权的方案中, G 的最小生成树的权值和之和为? ()



A. 50400

B. 53328

C. 63408

D. 80640

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 20, mod = 998244353;
5 int n, a[N][N], f[1 << N];
6
7 int main() {
8     cin >> n;
9     for (int i = 0; i < n; i++)
10         for (int j = 0; j < n; j++)
11             cin >> a[i][j];
12     f[0] = 1;
13     for (int i = 0; i < (1 << n); i++)
14         for (int j = 0; j < n; j++)
15             if (i >> j & 1)
16                 (f[i] += 1ll * f[i ^ (1 << j)] *
17                     a[__builtin_popcount(i) - 1][j] % mod) %= mod;
18     cout << f[(1 << n) - 1];
19     return 0;
20 }
```

保证输入的整数满足 $1 \leq n \leq 20, 1 \leq a_{i,j} < 998244353$ ，完成下面的判断题和单选题。

• 判断题

16. (1 分) 当 $n = 20$ 时，程序不会出现数组越界访问。()
17. 只要输入符合限定要求，无论输入的是什么，输出都一定是正整数。()
18. 删除第 16 行的 `1ll*`，不会对程序的输出产生任何影响。()

• 单选题

19. 当输入为 3 1 2 3 2 3 1 3 1 2 时，输出为 () ?
- A. 52 B. 53 C. 54 D. 55
20. 该代码的时间复杂度为 () ?
- A. $\Theta(n^2)$ B. $\Theta(2^n)$
- C. $\Theta(n \times 2^n)$ D. $\Theta(n^2 \times 2^n)$
21. 当 $n = 14$ 时，代码第 15 行的 `if` 语句中，表达式值为真的次数是 () ?
- A. 114688 B. 229176 C. 114588 D. 229376

(2)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 ll len[90], dp[90][2];
6
7 void init() {
8     len[0] = 1, len[1] = 2, dp[1][1] = 1;
9     for (int i = 2; i <= 86; i++) {
10         len[i] = len[i - 1] + len[i - 2];
11         int op = len[i - 1] & 1;
12         dp[i][0] = dp[i - 1][0] + dp[i - 2][0 ^ op];
13         dp[i][1] = dp[i - 1][1] + dp[i - 2][1 ^ op];
14     }
15 }
16
17 int solve1(int n, int p) {
18     string s = "0", nxt = "01", tmp;
19     while (nxt.length() < n)
20         tmp = nxt + s, s = nxt, nxt = tmp;
21     int ans = 0;
22     for (int i = p; i < n; i += 2) ans += nxt[i] - '0';
23     return ans;
24 }
25
26 int value(int pos) {
27     if (pos <= 1) return pos;
28     int k = 0;
29     while (len[k + 1] <= pos) k++;
30     return value(pos - len[k]);
31 }
32
33 int solve2(int n, int p) {
34     int ans = 0;
35     for (int i = p; i < n; i += 2) ans += value(i);
```


27. 若某次调用 `solve3(n, p)` 时, 语句 `ans += dp[k][p]` 恰好执行了 8 次, 则输入的 n 最小可能是多少? ()
- A. 986 B. 1596 C. 2583 D. 4180

(3)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 typedef unsigned long long ull;
6 const int N = 100005, M = 200005, S = 400037;
7 const int P = 998244353, iv2 = (P + 1) / 2;
8
9 int n, m, w[M], sum, dep[N], ct[M], sw[M], cur[M], ans[N];
10 ull h[N], he[M];
11 bool vis[N];
12 vector<pair<int, int> > G[N], T[N];
13 mt19937_64 rnd(random_device{}());
14
15 int tot, hd[S];
16 struct node { int nxt; ull key; } mp[M];
17 int get(ull key) {
18     int u = key % S;
19     for (int i = hd[u]; i; i = mp[i].nxt)
20         if (mp[i].key == key) return i;
21     return mp[++tot] = {hd[u], key}, hd[u] = tot;
22 }
23
24 void dfs1(int u, int p) {
25     vis[u] = 1;
26     for (auto [v, i] : G[u]) if (i ^ p) {
27         if (!vis[v])
28             dep[v] = dep[u] + 1, T[u].push_back({v, i}),
29             dfs1(v, i), h[u] ^= h[v], he[i] = h[v];
30         else if (dep[v] < dep[u])
31             he[i] = rnd(), h[u] ^= he[i], h[v] ^= he[i];
32     }
33 }
34
35 void dfs2(int u, ll c, ll b, ll b2) {
```



```

36     ans[u] = ((c + b * sum - (b * b + b2) % P * iv2) % P + P) % P;
37     for (auto [v, i] : T[u]) {
38         ll w = ::w[i];
39         if (!he[i]) {
40             dfs2(v, c, (b + w) % P, (b2 + w * w) % P);
41             continue;
42         }
43         int j = get(he[i]);
44         if (ct[j] == 1) dfs2(v, c, b, b2);
45         else {
46             int d = (w * (sw[j] - cur[j] * 2 - w) % P + P) % P;
47             cur[j] = (cur[j] + w) % P;
48             dfs2(v, (c + d) % P, b, b2);
49             cur[j] = (cur[j] - w + P) % P;
50         }
51     }
52 }
53
54 int main() {
55     ios::sync_with_stdio(0), cin.tie(0);
56     cin >> n >> m;
57     for (int i = 1, u, v; i <= m; i++)
58         cin >> u >> v >> w[i], sum = (sum + w[i]) % P,
59         G[u].push_back({v, i}), G[v].push_back({u, i});
60     dfs1(1, 0);
61     for (int i = 1, j; i <= m; i++) if (he[i])
62         j = get(he[i]), ct[j]++, sw[j] = (sw[j] + w[i]) % P;
63     dfs2(1, 0, 0, 0);
64     for (int i = 1; i <= n; i++) cout << ans[i] << ' ';
65 }

```

假设输入的整数满足 $3 \leq n \leq 10^5, 2 \leq m \leq 2 \times 10^5, 0 \leq w_i < 998244353$, 且输入的图连通、无自环或重边, 不考虑异或哈希值的冲突, 完成下面的判断题和单选题。

• 判断题

28. 代码中使用了哈希表, 攻击者可以在不获知随机数种子的前提下, 通过构造数据, 使得程序在该数据下的期望时间复杂度退化至平方级别。()

29. 若对于 $i \neq j$ 有 $he[i]$ 和 $he[j]$ 相等且均不为 0, 则图中存在一个环同时包含边 i, j 。
()
30. 若 $m = n - 1$, 且对于 $i < n$ 有 $u_i = i, v_i = i + 1, w_i = 1$, 则输出的所有数之和 (在对 P 取模意义下) 等于 $\frac{1}{3}n(n-1)(n-2)$ 。()

• 选择题

31. 这份程序对于每个点 u , 求得了 () 的答案对 P 取模的结果。
- A. 选择两条边 $i < j$, 满足存在一条 1 到 u 的路径 p , 使得 i, j 中至少有一条边在 p 上, 所有方案的 $w_i \times w_j$ 之和。
 - B. 选择两条边 $i < j$, 满足存在一条 1 到 u 的路径 p , 使得 i, j 都在 p 上, 所有方案的 $w_i \times w_j$ 之和。
 - C. 选择两条边 $i < j$, 满足对于所有 1 到 u 的路径 p , 都有 i, j 中至少有一条边在 p 上, 所有方案的 $w_i \times w_j$ 之和。
 - D. 选择两条边 $i < j$, 满足对于所有 1 到 u 的路径 p , 都有 i, j 都在 p 上, 所有方案的 $w_i \times w_j$ 之和。
32. 在 `dfs2` 函数中, 若执行了分支 `if (ct[j] == 1) dfs2(v, c, b, b2)`, 设当前正沿树边 e 向下遍历, 则 e 在原图中满足? ()
- A. 在原图中将边 e 删去后, 图的连通块数量必定会增加。
 - B. 在原图中将边 e 删去后, 图中必定会产生至少一条新的割边。
 - C. 边 e 在原图中必然不属于任何一个简单环。
 - D. 在原图中将边 e 删去后, 图中不会产生任何新的割边。
33. 对于如下输入数据, 输出的第 8 个数为? ()

```
10 10
1 2 10
2 3 20
3 4 30
4 5 40
5 6 10
6 7 10
7 8 10
8 9 10
9 10 10
10 5 10
```

- A. 6900 B. 9500 C. 10400 D. 16900

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1)(线性筛) 给定一个正整数 n ，要对于 $i = 1 \sim n$ 求出 i^i 取模 998244353 的结果。满足 $1 \leq n \leq 10^7$ 。你需要设计一个 $O(n)$ 的算法。

提示：在线性筛的过程中同步计算 $f_i = i^i \bmod 998244353$ 。质数可以直接使用快速幂；合数被筛到时，尝试将 f_{pq} 表示成 f_p 与 f_q 的幂的乘积。从而由已有结果完成转移。

令 $B = \lfloor \sqrt{n} \rfloor$ 。 f_p 的幂可将指数按 B 分块并预处理； f_q 的幂利用对于枚举的 q ，指数递增的性质，根据相邻质数之差增量维护。总时间复杂度即可达到 $O(n)$ 。

根据以上的提示，试补全程序。

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int N = 1e7+5, SN = (int)sqrt(N) + 5, mod = 998244353;
6 int qpow(int a, int b) {
7     int res = 1;
8     while (b) {
9         if (b & 1) res = 1ull * res * a % mod;
10        ①
11        b >>= 1;
12    }
13    return res;
14 }
15 int bsgs1[SN][SN], bsgs2[SN][SN];
16 bool vis[N];
17 int f[N], pr[N / 10], len;
18 int powers[N], S;
19 int main() {
20     int n;
21     scanf("%d", &n);
22     f[1] = 1;
23     const int B = sqrt(n);
24     for (int i = 2; i <= n; i++) {
25         if (!vis[i]) {
26             pr[++len] = i, f[i] = qpow(i, i);
```

```

27         if (i <= B) {
28             bsgs1[i][0] = 1;
29             for (int j = 1; j <= B; j++)
30                 bsgs1[i][j] = 1ull * bsgs1[i][j - 1] * f[i] % mod;
31             bsgs2[i][0] = 1;
32             for (int j = 1; j <= B; j++)
33                 ②
34         }
35     }
36     powers[0] = 1;
37     int cur = 1, gap = 0;
38     for (int j = 1; j <= len && i * pr[j] <= n; j++) {
39         vis[pr[j] * i] = 1;
40         int now = pr[j] - pr[j - 1];
41         ③
42         powers[ex] = 1ull * powers[ex - 1] * f[i] % mod;
43         gap = max(gap, now);
44         cur = 1ull * cur * powers[now] % mod;
45         ④
46         if (⑤)
47             break;
48     }
49 }
50 for (int i = 1; i <= n; i++)
51     printf("%d ", f[i]);
52 return 0;
53 }

```

34. ① 处应填 ()。

- A. `a = 1ull * a * a % mod;`
- B. `a = 1ull * a * b % mod;`
- C. `res = 1ull * a * a % mod;`
- D. `a = 1ull * res * res % mod;`

35. ② 处应填 ()。

- A. `bsgs2[i][j] = 1ull * bsgs2[i][j - 1] * qpow(i, B) % mod;`
- B. `bsgs2[i][j] = 1ull * bsgs1[i][j - 1] * bsgs2[i][B - 1] % mod;`

- C. `bsgs2[i][j] = 1ull * bsgs1[i][j - 1] * qpow(f[i], B) % mod;`
- D. `bsgs2[i][j] = 1ull * bsgs2[i][j - 1] * bsgs1[i][B] % mod;`

36. ③ 处应填 ()。

- A. `for(int ex = 1; ex <= now; ex++)`
- B. `for(int ex = gap + 1; ex <= now; ex++)`
- C. `for(int ex = 1; ex <= now; ex += B)`
- D. `for(int ex = gap + 1; ex <= now; ex += B)`

37. ④ 处应填 ()。

- A. `f[pr[j] * i] = 1ull * bsgs2[pr[j] * i][i % B] *
bsgs1[pr[j] * i][i / B] % mod * cur % mod;`
- B. `f[pr[j] * i] = 1ull * bsgs1[pr[j]][i % B] *
bsgs2[pr[j]][i / B] % mod * cur % mod;`
- C. `f[pr[j] * i] = 1ull * bsgs2[pr[j]][i % B] *
bsgs1[pr[j]][i / B] % mod * cur % mod;`
- D. `f[pr[j] * i] = 1ull * bsgs1[pr[j] * i][i % B] *
bsgs2[pr[j] * i][i / B] % mod * cur % mod;`

38. ⑤ 处应填 ()。

- A. `pr[j] > i`
- B. `!(pr[j] % i)`
- C. `!(i % pr[j])`
- D. `i > pr[j]`

(2) (静态 Top Tree)

给定一棵由主链和若干叶子组成的带权树。主链包含顶点 $1, 2, \dots, n$, 对于 $1 \leq i < n$, 顶点 i 与顶点 $i+1$ 之间有一条长度为 a_i 的边。每个主链顶点还可能连接任意多条叶边; 所有叶边按照输入顺序编号为 $1, 2, \dots, k$ 。共有 m 次操作, 每次操作为以下三种之一:

- $1 \ x \ y$: 将主链边 a_x 的长度修改为 y ;
- $2 \ x \ y$: 将编号为 x 的叶边长度修改为 y ;
- $3 \ l \ r$: 询问由主链顶点 $l, l+1, \dots, r$ 以及与它们相连的所有叶子组成的子树的直径长度。

其中, $2 \leq n \leq 10^5$, $0 \leq k \leq 10^5$, $1 \leq m \leq 10^5$, 所有边长均为不超过 10^9 的非负整数。输入保证所有操作均合法。

输入的第一行包含三个整数 n, k, m 。第二行包含 $n-1$ 个整数 $a_1, a_2, \dots, a_{n-1}$ 。接下来的 n 行中, 第 i 行首先包含整数 c_i , 随后包含 c_i 个整数, 依次表示与顶点 i 相连的叶边长度; 保证 $\sum c_i = k$ 。最后 m 行每行包含一次操作。

为解决该问题, 可以建立一棵静态 Top Tree。Top Tree 中的每个结点表示原树的一个连通子图, 称为簇。一个顶点若属于该簇, 同时还与不属于该簇的树边相连, 则称为边界顶点。每个簇至多有两个边界。

静态 Top Tree 使用 `rake(R)` 和 `compress(C)` 两种合并。对于簇 x , 记 $E(x)$ 为簇内边的集合, $V(x)$ 为边界顶点集合。边集不相交的簇 a, b 只有在恰有一个公共边界, 即 $|V(a) \cap V(b)| = 1$ 时才能合并, 以保证结果仍为簇。`rake` 要求 b 只有一个边界, `compress` 要求 a, b 均有两个边界。若 r, c 分别为两种合并的结果, 则 $E(r) = E(c) = E(a) \cup E(b)$, $V(r) = V(a), V(c) = (V(a) \cup V(b)) \setminus (V(a) \cap V(b))$ 。其中 $A \setminus B$ 表示所有属于集合 A 但不属于集合 B 的元素组成的集合。

Top Tree 上每个顶点代表的簇都是两个儿子顶点的簇以 `R` 或 `C` 方式合并得到。在本题中, 算法先平衡合并每个主链顶点的叶簇, 再将分支簇挂到对应主链边的左端, 并在线段树中合并主链边簇。这样建立的 Top Tree 就可以在树上取出一个区间的簇回答询问。实现上, 令簇信息 `Node` 维护边界距离 `w`、从左右边界出发的最远距离 `l` 和 `r`、簇内直径 `d` 及边界个数 `c`, 即可实现合并。

以上算法的预处理复杂度为 $O(n+k)$; 每次操作复杂度为 $O(\log n + \log k)$ 。

根据以上的提示, 试补全程序。

```
1 #include <algorithm>
2 #include <iostream>
3 using namespace std;
4 const int N = 100005, M = 200005;
5 int n, k, m, z, tot, st[N], bel[N], id[N], lc[M], rc[M], fa[M];
```

```

6 long long a[N];
7 struct Node {
8     long long w, l, r, d;
9     int c;
10 } f[M], t[N << 2];
11 Node leaf(long long x) { return {0, x, x, x, 1}; }
12 Node path(long long x) { return {x, x, x, x, 2}; }
13 Node merge(Node a, Node b, char o, int s = 0) {
14     Node z = a;
15     if (o == 'R') {
16         if (a.c == 1) {
17             z.l = z.r = max(a.l, b.l);
18         } else if (s == 0) {
19             z.l = max(a.l, b.l); z.r = max(a.r, a.w + b.l);
20         } else {
21             z.l = max(a.l, a.w + b.l); z.r = max(a.r, b.l);
22         }
23         z.d = max(max(a.d, b.d), ① );
24     } else {
25         z.w = a.w + b.w; z.l = max(a.l, a.w + b.l);
26         z.r = max(b.r, b.w + a.r);
27         z.d = max(max(a.d, b.d), a.r + b.l); z.c = 2;
28     }
29     return z;
30 }
31 int join(int x, int y) {
32     int u = ++tot; lc[u] = x; rc[u] = y; fa[x] = fa[y] = u;
33     f[u] = merge(f[x], f[y], 'R'); return u;
34 }
35 int build_rake(int l, int r) {
36     if (l == r) return l;
37     int mid = (l + r) >> 1;
38     return join(build_rake(l, mid), build_rake(mid + 1, r));
39 }
40 Node star(int x) { return st[x] ? f[st[x]] : leaf(0); }

```

```

41 Node make(int x) {
42     Node y = path(a[x]);
43     if (st[x]) ② ;
44     return y;
45 }
46 void build(int x, int l, int r) {
47     if (l == r) { t[x] = make(l); return; }
48     int mid = (l + r) >> 1;
49     build(x << 1, l, mid); build(x << 1 | 1, mid + 1, r);
50     t[x] = merge(t[x << 1], t[x << 1 | 1], 'C');
51 }
52 void change(int x, int l, int r, int q) {
53     if (l == r) { t[x] = make(l); return; }
54     int mid = (l + r) >> 1;
55     if (q <= mid) change(x << 1, l, mid, q);
56     else change(x << 1 | 1, mid + 1, r, q);
57     t[x] = merge(t[x << 1], t[x << 1 | 1], 'C');
58 }
59 Node query(int x, int l, int r, int ql, int qr) {
60     if (ql <= l && r <= qr) return t[x];
61     int mid = (l + r) >> 1;
62     if (qr <= mid) return query(x << 1, l, mid, ql, qr);
63     if (ql > mid) return query(x << 1 | 1, mid + 1, r, ql, qr);
64     Node p = query(x << 1, l, mid, ql, qr);
65     Node q = query(x << 1 | 1, mid + 1, r, ql, qr);
66     return ③;
67 }
68 void modify(int x, long long y) {
69     int u = id[x]; f[u] = leaf(y);
70     while (fa[u]) { u = fa[u]; f[u] = merge(f[lc[u]], f[rc[u]], 'R'); }
71     if (④) change(1, 1, n - 1, bel[x]);
72 }
73 int main() {
74     cin >> n >> k >> m;
75     for (int i = 1; i < n; i++) cin >> a[i];

```



```

76     for (int i = 1; i <= n; i++) {
77         int c, l = tot + 1; cin >> c;
78         while (c--) {
79             long long x; cin >> x; bel[++z] = i; id[z] = ++tot;
80             f[tot] = leaf(x);
81         }
82         if (l <= tot) st[i] = build_rake(l, tot);
83     }
84     build(1, 1, n - 1);
85     while (m--) {
86         int o, x, y; cin >> o >> x >> y;
87         if (o == 1) { a[x] = y; change(1, 1, n - 1, x); }
88         else if (o == 2) modify(x, y);
89         else {
90             Node ans = ⑤;
91             if (x < y) ans = merge(ans, star(y), 'R', 1);
92             cout << ans.d << endl;
93         }
94     }
95     return 0;
96 }

```

39. ① 处应填 ()。

- A. (a.c == 1 || s == 1 ? a.l : a.r) + b.l
- B. (a.c == 1 ? max(a.d, b.d) : (s == 0 ? a.l : a.r)) + b.l
- C. (a.c == 1 || s == 0 ? a.l : a.w) + b.l
- D. (a.c == 1 || s == 0 ? a.l : a.r) + b.l

40. ② 处应填 ()。

- A. y = merge(y, star(x), 'R', 0);
- B. y = merge(y, star(x), 'R', 1);
- C. y = merge(star(x), y, 'R', 0);
- D. y = merge(y, star(x), 'C');

41. ③ 处应填 ()。

- A. merge(q, p, 'C')
- B. merge(p, q, 'R', 0)
- C. merge(p, q, 'R', 1)

D. merge(p, q, 'C')

42. ④ 处应填 ()。

A. bel[x] < n

B. x < n

C. bel[x] <= n

D. st[bel[x]] != 0

43. ⑤ 处应填 ()。

A. x == y ? leaf(0) : query(1, 1, n - 1, x, y - 1)

B. x == y ? star(x) : query(1, 1, n - 1, x, y)

C. x == y ? star(x) : query(1, 1, n - 1, x, y - 1)

D. x == y ? star(x) : query(1, 1, n - 1, x + 1, y - 1)